

II.3 Couche IoT — Acquisition et Communication

II.3.1 Analyse des besoins de mesure

Contexte réglementaire

La surveillance de la pollution industrielle en Tunisie est encadrée par le **décret gouvernemental n°2018-928 du 7 novembre 2018**, modifiant et complétant le décret n°2010-2519 du 28 septembre 2010, fixant les valeurs limite à la source des polluants de l'air des sources fixes. Le projet cible le site industriel de Sfax, un pôle industriel dense où la concentration d'émissions polluantes constitue un risque sanitaire documenté.

Le système de surveillance est conçu comme une **plateforme générique** applicable à tout type d'industrie tunisienne. Les valeurs limites d'émission (VLE) retenues sont donc les **valeurs générales de l'Annexe 1** du décret, applicables à toutes sources fixes industrielles en l'absence de valeur spécifique plus contraignante. Ces valeurs constituent le socle réglementaire minimal commun. Pour un déploiement sur un secteur spécifique (cimenterie → Annexe 6, raffinerie → Annexe 2, etc.), le modèle `ThresholdConfig` du backend permet de surcharger ces seuils sans modification du code.

Les polluants surveillés ont été choisis selon trois critères : leur présence significative dans les émissions industrielles tunisiennes, leur couverture par le décret, et la disponibilité locale de capteurs adaptés à un déploiement embarqué.

Tableau des polluants ciblés

Polluant	Formule	Unité système	VLE (Décret 2010-2519, Annexe 1 — valeurs générales)	Capteur retenu	Plage capteur
Dioxyde de carbone	CO ₂	ppm	Non réglementé (seuil interne KPI)	MH-Z19B	0 – 5 000 ppm
Dioxyde d'azote	NO ₂	mg/Nm ³	500 mg/Nm³ (flux > 25 kg/h)	MQ-131	0 – 20 ppm → ~41 mg/Nm ³ (mesure NO _x +O ₃)
Dioxyde de soufre	SO ₂	mg/Nm ³	300 mg/Nm³ (flux > 25 kg/h)	MQ-136	0 – 200 ppm → ~572 mg/Nm ³ max
Composés organiques volatils	COV / TVOC	mg/Nm ³	110 mg/Nm³ (flux > 2 kg/h)	SGP30 (I ² C)	0 – ~60 mg/Nm ³
Particules totales	PM _{2.5}	mg/m ³	40 mg/m³ (flux > 1 kg/h)	SDS011	0 – 999 µg/m ³
Particules totales	PM ₁₀	mg/m ³	40 mg/m³ (flux > 1 kg/h)	SDS011	0 – 999 µg/m ³
Température ambiante	—	°C	Paramètre de contexte	DHT22	-40 – +80°C
Humidité relative	—	%RH	Paramètre de contexte	DHT22	0 – 100 %RH

Note : Les VLE de l'Annexe 1 sont des valeurs générales applicables à toutes sources fixes industrielles. Elles constituent les seuils par défaut du système. Pour un déploiement sectoriel spécifique, le modèle `ThresholdConfig` (MongoDB) permet de les remplacer par les valeurs de l'annexe réglementaire appropriée (ex. Annexe 6 pour les cimenteries). Le backend stocke ces valeurs dans le champ `regulatoryLimit` du modèle `Polluant` ; le moteur d'alertes (`ReadingService.js`) s'appuie dessus pour déclencher les alertes.

Les MQ-136 et MQ-131 mesurent en **ppm** via la courbe Rs/R0 du datasheet ; le firmware applique la conversion en mg/Nm³ avant publication MQTT (×2,86 pour SO₂, ×2,05 pour NO_x), assurant la cohérence directe avec les seuils stockés en base.

II.3.2 Choix de la carte microcontrôleur : ESP32-WROOM-32

Justification technique

L'ESP32 DevKit V1 (module WROOM-32) a été retenu comme unité centrale de la station de surveillance. Ce choix repose sur un ensemble de caractéristiques qui répondent directement aux contraintes du projet.

Capacités WiFi intégrées. Le module intègre nativement un émetteur-récepteur WiFi 802.11 b/g/n sur la bande 2,4 GHz, éliminant tout module réseau externe et simplifiant le câblage. La connectivité WiFi permet la publication MQTT directe vers le broker Mosquitto hébergé sur le serveur backend.

Double cœur 240 MHz. L'architecture Xtensa LX6 dual-core à 240 MHz permet de séparer la gestion réseau (WiFi/MQTT, exécutée sur le cœur 0 par le stack ESP-IDF) des tâches applicatives (lecture capteurs, construction JSON). Cela garantit que les reconnexion réseau ne bloquent pas l'acquisition de données.

520 KB de RAM. La mémoire SRAM intégrée est suffisante pour maintenir les buffers UART des capteurs (MH-Z19B, SDS011), le document JSON ArduinoJson (384 octets par message), et le stack WiFi.

Support MQTT natif. La bibliothèque `PubSubClient` pour Arduino/ESP32 fournit une implémentation MQTT complète, compatible avec le broker Mosquitto utilisé en production. Le firmware configure un buffer de 512 octets (`mqtt.setBufferSize(512)`) adapté aux payloads JSON du projet.

Richesse des interfaces. L'ESP32 expose simultanément deux UART matériels (Serial1, Serial2), un bus I²C, et plusieurs canaux ADC 12 bits — exactement ce que requiert le tableau de capteurs hétérogènes du projet.

Faible coût et disponibilité locale. Le DevKit V1 est disponible chez les revendeurs électroniques de Sfax pour moins de 15 DT, en accord avec les contraintes budgétaires du projet.

Tableau comparatif des alternatives

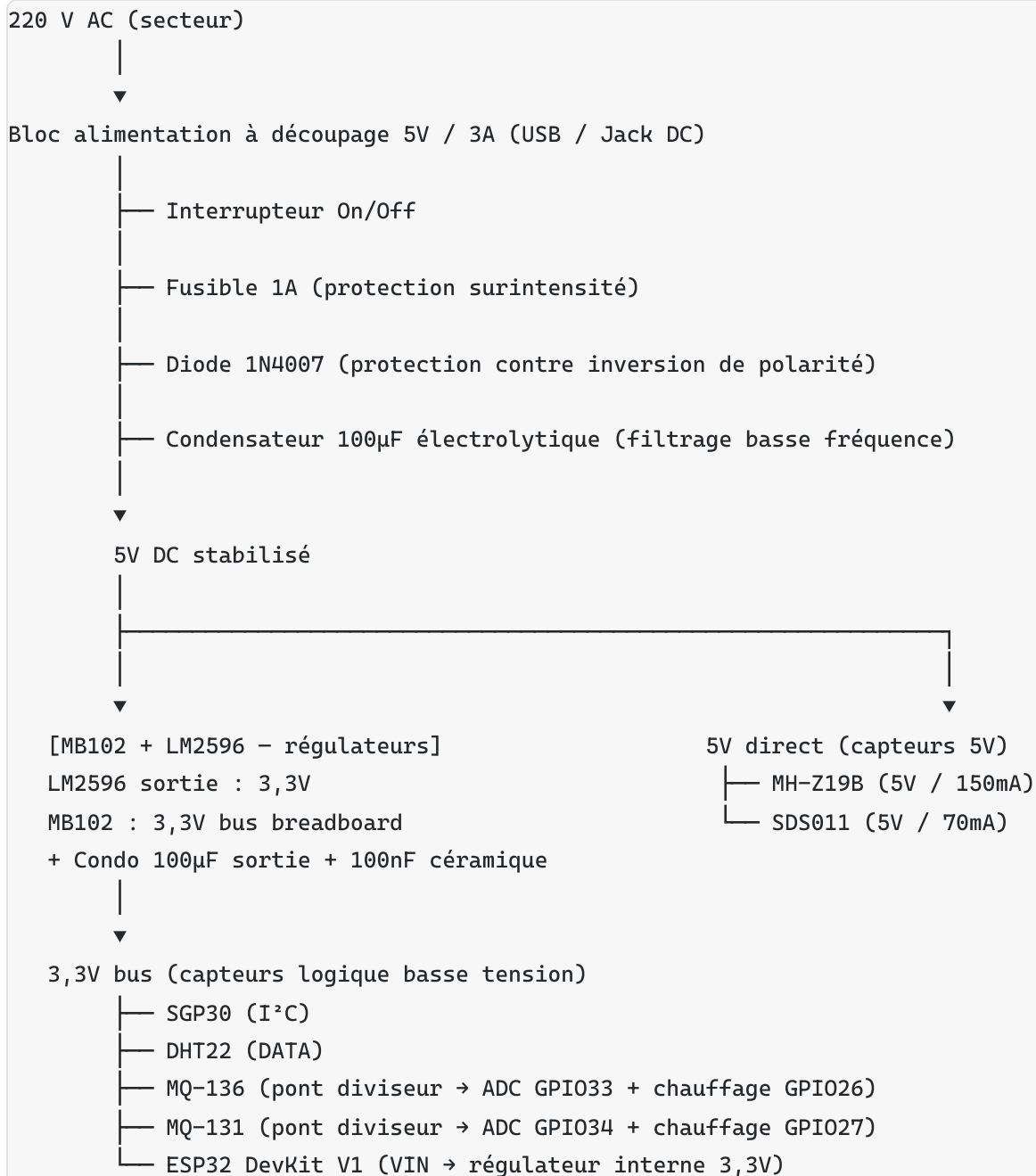
Critère	ESP32 DevKit V1 ✓	Arduino Mega 2560	Raspberry Pi Pico W
WiFi intégré	Oui (802.11 b/g/n)	Non (shield externe requis)	Oui (CYW43439)
Fréquence CPU	240 MHz dual-core	16 MHz single-core	133 MHz dual-core
RAM	520 KB SRAM	8 KB SRAM	264 KB SRAM
UART matériels	3 (Serial0/1/2)	4	2
ADC résolution	12 bits (ADC1 — WiFi safe)	10 bits	12 bits
Support MQTT	PubSubClient natif	Limité	Bibliothèque Python
Environnement dev	PlatformIO / Arduino	Arduino IDE	MicroPython / C SDK
Coût estimé (TN)	~15 DT	~25 DT	~18 DT
Verdict	Retenu	Insuffisant (RAM, WiFi)	Alternative possible mais écosystème C++ limité

Conclusion : L'Arduino Mega est écarté principalement pour son manque de connectivité WiFi native et ses 8 KB de RAM, insuffisants pour le stack TCP/IP. Le Raspberry Pi Pico W constitue une alternative valable mais son écosystème C++ embarqué est moins mature que celui de l'ESP32 pour ce type d'application.

Contraintes d'alimentation retenues

L'alimentation de la station est assurée par un **bloc d'alimentation à découpage 5 V / 3 A** (format USB ou Jack DC) branché directement sur le secteur 220 V AC disponible dans les bâtiments industriels. Il n'y a pas de transformateur linéaire intermédiaire : le bloc secteur intègre lui-même la conversion AC/DC.

La distribution interne de l'énergie repose sur deux éléments complémentaires : un **module MB102** (Power Supply Module pour breadboard), qui distribue simultanément le 5 V et le 3,3 V à partir d'une entrée USB ou Jack DC, et un **régulateur buck LM2596** utilisé pour les charges nécessitant un 3,3 V stable avec une meilleure capacité de courant. La chaîne complète est la suivante :



Condensateurs de découplage. Un condensateur de **100 µF électrolytique** est placé en entrée/sortie du LM2596, et un **condensateur céramique 100 nF** est positionné au plus près de chaque circuit intégré (ESP32, SGP30) pour filtrer les transitoires haute fréquence. Ces valeurs sont spécifiées dans le schéma électronique du projet (`SCHEMA_ELECTRONIQUE.svg`).

Le champ `battery` dans le payload MQTT est systématiquement à `null` (firmware `main.cpp` , `doc["battery"] = nullptr`), confirmant l'alimentation permanente sur secteur.

II.3.3 Sélection et justification des capteurs

MH-Z19B — Dioxyde de carbone (CO₂)

Principe physique. Le MH-Z19B utilise la technologie **NDIR** (Non-Dispersive InfraRed). Une source infrarouge émet un rayonnement dans la cellule de mesure ; les molécules de CO₂ absorbent spécifiquement la longueur d'onde 4,26 µm. La concentration est déduite de l'atténuation du signal mesuré par le détecteur. Cette méthode est hautement sélective au CO₂ et insensible à l'humidité et aux gaz interférants.

Caractéristiques techniques :

- Plage de mesure : 0 – 5 000 ppm
- Précision : ± 50 ppm ± 5 % de la lecture
- Interface : UART 9600 bauds, 8N1 (Serial2 sur ESP32)
- Alimentation : 5 V DC, courant pic ≈ 150 mA
- Temps de préchauffage : 3 minutes (correction ABC automatique)
- Durée de vie : > 5 ans

Justification du choix. La mesure NDIR est la seule technologie portable capable d'identifier spécifiquement le CO₂ à des concentrations industrielles. La fonctionnalité ABC (Automatic Baseline Correction) compense la dérive à long terme sans intervention manuelle, ce qui est critique pour une station de surveillance autonome.

Câblage. Le MH-Z19B est alimenté en 5 V. Ses signaux TX/RX transitent par un **convertisseur de niveau bidirectionnel** avant les GPIO 16/17 de l'ESP32 (3,3 V) pour éviter d'endommager les entrées logiques.

Dans le firmware : Initialisé via `mhz19.begin(Serial2)` avec `mhz19.autoCalibration(true)`. La lecture se fait via `readCo2(ppm)` qui vérifie que la valeur retournée est > 0 (valeur 0 = erreur UART ou capteur non prêt).

Fournisseur local. CoThings, Sfax.

MQ-136 — Dioxyde de soufre (SO₂)

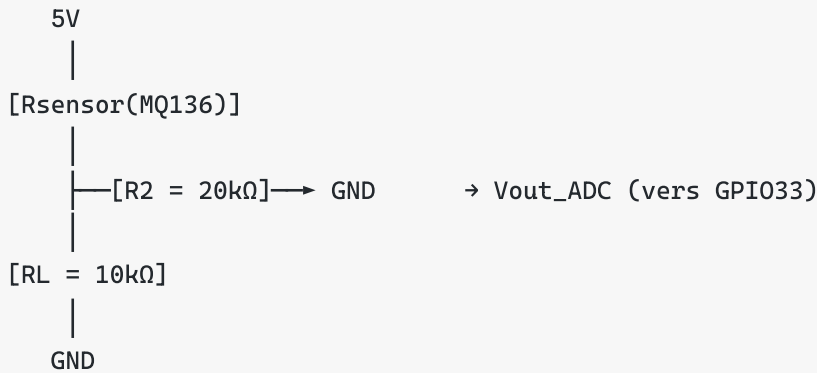
Principe physique. Le MQ-136 (Hanwei Electronics) est un capteur à **oxyde métallique (MOS)**. Un élément sensible en SnO₂ est chauffé à ~200 °C par un filament résistif intégré. En présence de SO₂, la résistance de surface du matériau diminue de manière proportionnelle à la concentration selon une loi log-linéaire. Ce type de capteur est robuste, peu coûteux et disponible localement.

Caractéristiques techniques :

- Gaz cible principal : SO₂ (sensible aussi à H₂S, NH₃)
- Plage de mesure : 1 – 200 ppm
- Tension de chauffage : 5 V DC (~150–200 mA)
- Tension de circuit : 5 V DC (pont diviseur + RL = 10 kΩ)

- Temps de réponse T90 : < 60 s
- Temps de préchauffe : $\geq 24\text{--}48$ h pour stabilisation R0
- Interface : sortie analogique (tension) $\rightarrow$ diviseur résistif $\rightarrow$ ADC ESP32

Interface avec l'ESP32 — pont diviseur résistif. La sortie du MQ-136 est une tension entre 0 et 5 V (pont Rsensor + RL). Un **diviseur résistif** ($R1 = 10\text{ k}\Omega$, $R2 = 20\text{ k}\Omega$) ramène cette tension à $\leq 3,3\text{ V}$ pour l'ADC ESP32 :



La conversion tension $\rightarrow$ ppm est effectuée dans `sensors.cpp` via la courbe du datasheet :

```
// Rs = RL × (Vcc / Vout_reel - 1)
// ppm = MQ136_A × (Rs / R0) ^ MQ136_B    (courbe log-linéaire Hanwei)
// value = ppm × 2.86    (conversion  $\rightarrow$  mg/Nm³)
```

Justification du choix. Le MQ-136 est un capteur de la famille MOS disponible chez tous les distributeurs électroniques tunisiens (~8 DT). Bien que moins précis et moins sélectif que les capteurs électrochimiques, il convient parfaitement à un prototype PFE : circuit simple (pas de TIA), tension de sortie directe, bibliothèque Arduino disponible. La calibration de R0 en air propre constitue une partie expérimentale documentable du travail.

Limitation. La mesure est semi-quantitative sans calibration terrain. La sensibilité croisée (H_2S , NH_3) peut générer de faux positifs dans certains environnements industriels. Ces limitations sont documentées dans les perspectives d'amélioration du projet.

Câblage. Le MQ-136 est alimenté en 5 V. Le GPIO de chauffage (GPIO26) est contrôlé par un transistor NPN 2N2222 (résistance de base $1\text{ k}\Omega$, diode 1N4007 de protection). La sortie analogique passe par le diviseur résistif avant **GPIO33** (ADC1 CH5).

Fournisseur local. Disponible chez tous les revendeurs électroniques Sfax/Tunis (~8 DT).

MQ-131 — Oxydes d'azote (NO_x)

Principe physique. Identique au MQ-136 : capteur MOS à base de SnO_2 . Le MQ-131 est optimisé pour la détection des **oxydants puissants** : NO_x (NO_2 , NO) et ozone (O_3). Sa résistance de surface diminue en présence de ces gaz selon une courbe log-linéaire. Il n'est pas sélectif entre NO_2 et O_3 — la mesure représente une **concentration d'oxydants totaux**.

Caractéristiques techniques :

- Gaz cibles : NO₂, NO (NO_x) + O₃ (mesure combinée)
- Plage de mesure : 10 ppb – 200 ppm
- Tension de chauffage : 5 V DC (~150–200 mA)
- Tension de circuit : 5 V DC
- Temps de réponse T90 : < 60 s
- Temps de préchauffe : ≥ 24–48 h pour calibration R0
- Interface : sortie analogique → diviseur résistif → ADC ESP32

Interface sur GPIO34. Même architecture diviseur résistif que le MQ-136. GPIO34 est une broche **input-only** de l'ESP32 (ADC1 CH6), adaptée à cette lecture analogique. Le chauffage est contrôlé via GPIO27.

```
// Conversion MQ-131 dans sensors.cpp :  
// Rs = RL × (Vcc / Vout_reel - 1)  
// ppm = MQ131_A × (Rs / R0) ^ MQ131_B  
// value = ppm × 2.05    (→ mg/Nm³, facteur NO₂)
```

Justification du choix. Pour un prototype PFE à bas coût, le MQ-131 offre une détection de la présence de composés oxydants azotés dans l'environnement industriel. Son prix (~10 DT) et sa disponibilité locale le rendent adapté à la phase de prototypage. La non-sélectivité NO₂/O₃ est une limitation connue et documentée.

Limitation. La réponse combinée NO_x+O₃ ne permet pas de dissocier les deux espèces. La VLE réglementaire du décret (500 mg/Nm³) est définie pour les NO_x seuls (hors N₂O). La valeur publiée est donc indicative des oxydants totaux en zone périmétrique.

Câblage. Alimentation 5 V, chauffage via GPIO27/transistor NPN, sortie analogique via diviseur résistif vers **GPIO34** (ADC1 CH6, input-only).

Fournisseur local. Disponible chez tous les revendeurs électroniques Sfax/Tunis (~10 DT).

SGP30 — Composés organiques volatils (COV / TVOC)

Principe physique. Capteur **électrochimique I²C** (Sensirion). La mesure TVOC (Total Volatile Organic Compounds) est basée sur la variation de résistance d'une couche d'oxyde métallique chauffée en présence de COV. Le SGP30 intègre également un algorithme de correction d'humidité et produit un index de qualité d'air.

Caractéristiques techniques :

- Mesures : TVOC (ppb) et eCO₂ (ppm équivalent)
- Interface : I²C, adresse 0x58, SDA GPIO 21 / SCL GPIO 22
- Alimentation : 3,3 V exclusivement
- Période de mesure recommandée : ≥ 1 s (utilisé à 30 s dans le projet)

Conversion dans le firmware. Le SGP30 restitue le TVOC en ppb. Le backend attend des mg/Nm³. Le firmware applique une conversion approximative :

```
out.value = tvocPpb * 0.0045f; // ≈ ppb → mg/Nm³ (approximation)
out.rawValue = tvocPpb;        // ppb conservé comme valeur brute
```

Cette conversion est une approximation linéaire valable pour un mélange COV typique. Pour une mesure réglementaire certifiée, une calibration labo avec gaz étalon est requise.

Justification. Le SGP30 est le seul capteur COV du projet qui ne nécessite ni cycle de chauffe externe, ni circuit analogique complexe. Son interface I²C s'intègre directement sur le bus existant (SGP30 + OLED SSD1306).

SDS011 — Particules PM2.5 et PM10

Principe physique. Le SDS011 utilise la **diffusion laser** (OPC — Optical Particle Counter). Un laser 650 nm éclaire les particules en suspension ; la lumière diffusée est analysée par un photorécepteur. La granulométrie (PM2.5 et PM10) est déduite de l'intensité et de la fréquence des impulsions. Un ventilateur intégré aspire l'air à travers la chambre de mesure.

Caractéristiques techniques :

- Plages : PM2.5 : 0 – 999,9 µg/m³ | PM10 : 0 – 999,9 µg/m³
- Précision : ± 15 % (0 – 35 µg/m³), ± 10 % (35 – 500 µg/m³)
- Interface : UART 9600 bauds (Serial1 : RX GPIO 18 / TX GPIO 19)
- Mode d'acquisition : **Query mode** — l'ESP32 envoie une commande, le SDS011 répond avec une trame 10 octets contenant PM2.5 et PM10
- Durée de vie laser : ≈ 8 000 heures

Justification. La diffusion laser offre une mesure simultanée des deux fractions granulométriques avec une seule acquisition UART. Le mode query évite le flux continu qui saturerait le buffer UART.

Fournisseur local. 2btrading, Tunis (livraison Sfax).

Dans le firmware : Une seule lecture `readParticulates()` produit deux publications MQTT distinctes : `emissions/Zone-A/PM25` et `emissions/Zone-A/PM10`.

DHT22 — Température et humidité

Note sur le plan initial : Le SHT31 (I²C, ±0.3°C) a été envisagé initialement, mais remplacé par le **DHT22**, disponible immédiatement chez tous les distributeurs électroniques tunisiens. Son interface 1-wire n'occupe pas le bus I²C (déjà utilisé par SGP30 + OLED).

Caractéristiques DHT22 :

- Précision température : $\pm 0,5^{\circ}\text{C}$ (plage -40 à $+80^{\circ}\text{C}$)
- Précision humidité : $\pm 2 - 5\ \% \text{RH}$ (plage $0 - 100\ \% \text{RH}$)
- Interface : protocole 1-wire propriétaire, GPIO 4 (pull-up $10\ \text{k}\Omega$)
- Alimentation : $3,3\ \text{V} - 5\ \text{V}$

Rôle dans le système. La température et l'humidité sont publiées comme paramètres de contexte environnemental. Ils alimentent les modèles IA du backend pour la modélisation de la dispersion des polluants. La température est également un facteur de correction pour la sensibilité des capteurs électrochimiques ME4 (compensation recommandée en calibration avancée).

Afficheur OLED 128×64 — Retour opérateur local

Un afficheur OLED 128×64 pixels (contrôleur SSD1306, interface I²C) est prévu sur le même bus que le SGP30 pour afficher en temps réel les dernières valeurs mesurées et l'état de la connexion MQTT. Ce composant permet au technicien de maintenance de vérifier le fonctionnement de la station sans accès au tableau de bord web.

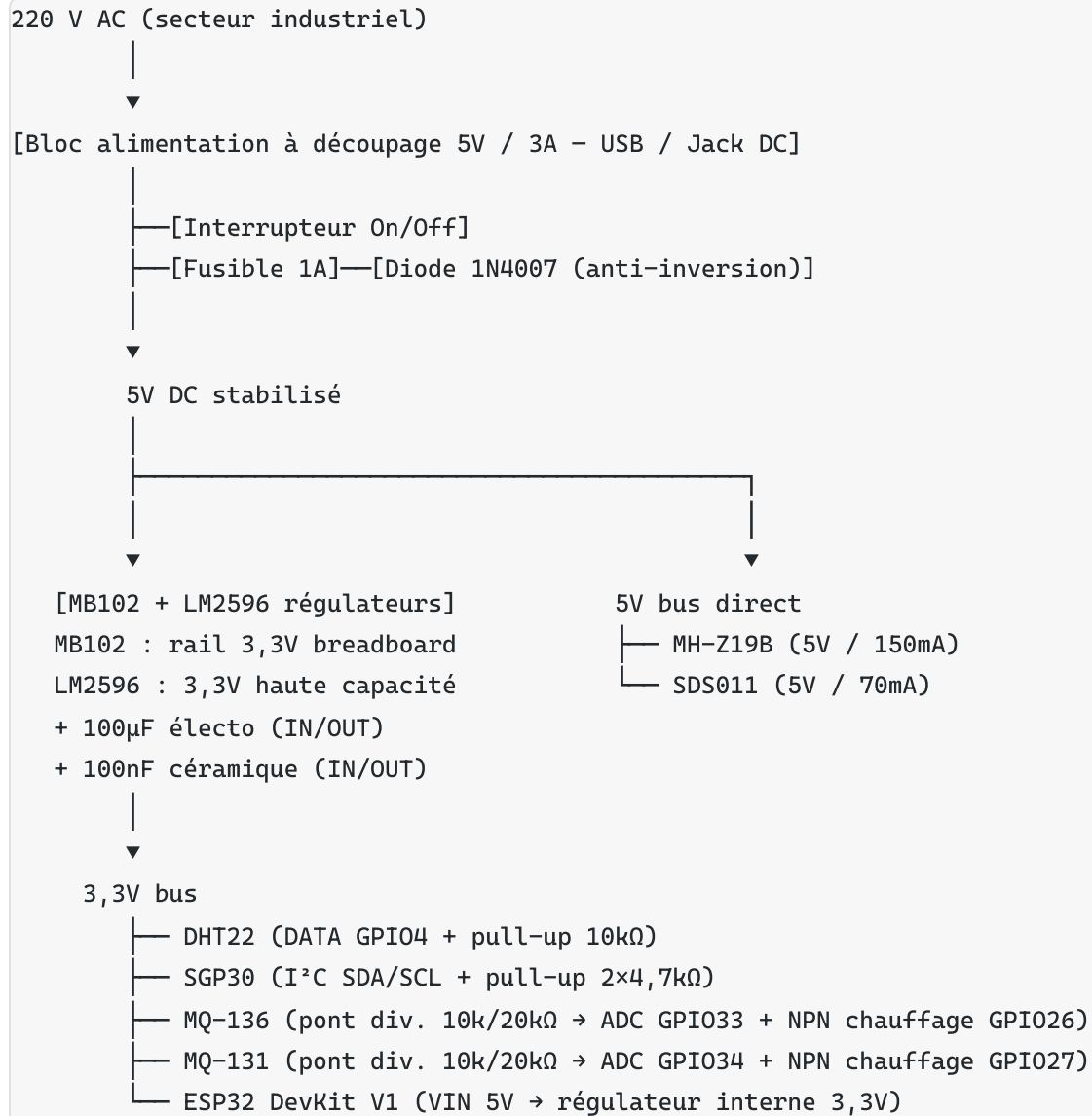
Tableau récapitulatif des capteurs

Capteur	Polluant mesuré	Principe	Interface GPIO	Fréquence	Alimentation	Notes câblage
MH-Z19B	CO ₂	NDIR infrarouge	UART — RX:16 TX:17	10 s	5 V / 150 mA	Level shifter bidirectionnel
MQ-136	SO ₂	Oxyde métallique (MOS)	ADC GPIO 33 (pont divis. RL=10kΩ)	30 s	5 V / 150–200 mA	Diviseur 10kΩ/20kΩ chauffage GPIO26
MQ-131	NO _x / O ₃	Oxyde métallique (MOS)	ADC GPIO 34 input-only (pont divis.)	30 s	5 V / 150–200 mA	Diviseur 10kΩ/20kΩ chauffage GPIO27
SGP30	COV / TVOC	MOX I ² C	I ² C SDA:21 SCL:22 + pull-up 2×4,7kΩ	30 s	3,3 V	Bus partagé avec OLE
SDS011	PM2.5 / PM10	Diffusion laser	UART — RX:18 TX:19	15 s	5 V / 70 mA	Level shifter bidirectionnel
DHT22	Température / Humidité	Capacitif 1-wire	GPIO 4 + pull-up 10 kΩ	10 s	3,3 V	—



II.3.4 Architecture matérielle du nœud IoT

Schéma d'alimentation



Affectation complète des GPIO

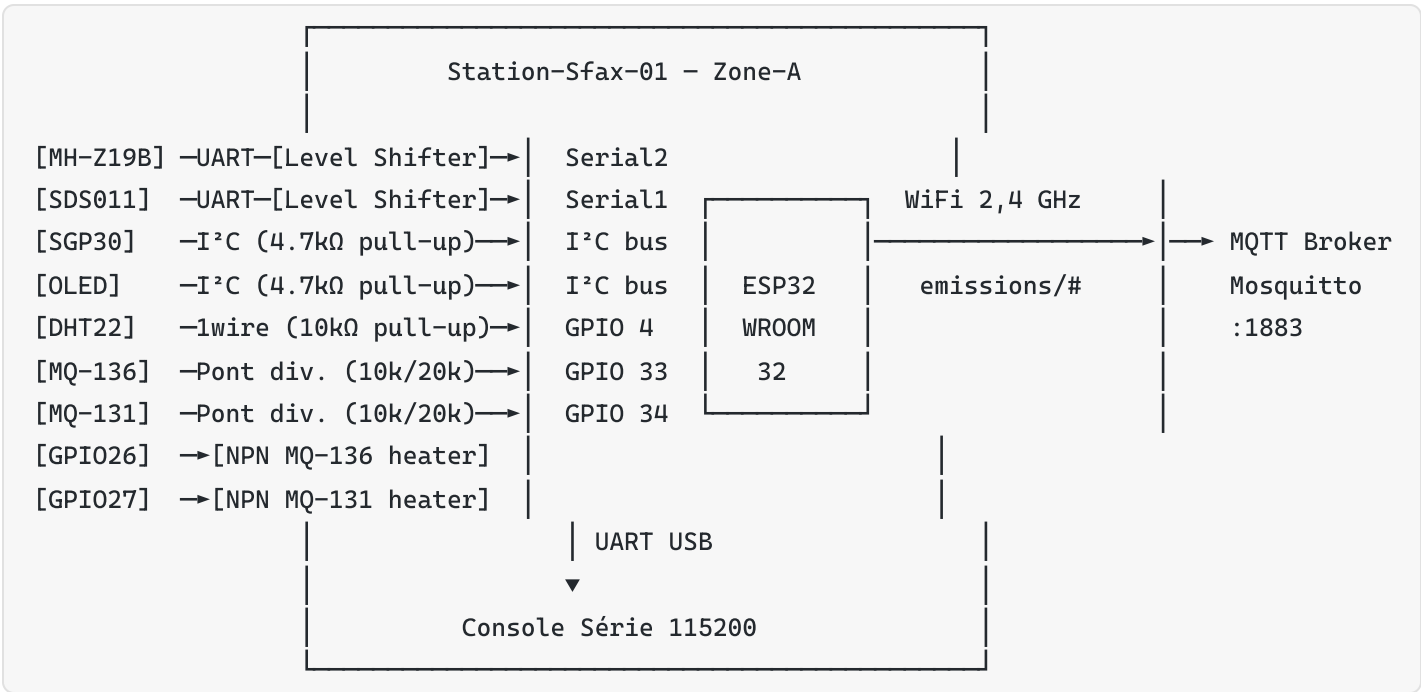
ESP32 DevKit V1

GPIO 4	←	DHT22 DATA (pull-up 10 kΩ vers 3,3V)
GPIO 16	←	MH-Z19B TX (UART RX – Serial2)
GPIO 17	→	MH-Z19B RX (UART TX – Serial2)
GPIO 18	←	SDS011 TX (UART RX – Serial1)
GPIO 19	→	SDS011 RX (UART TX – Serial1)
GPIO 21	↔	I ² C SDA (SGP30 + OLED SSD1306)
GPIO 22	↔	I ² C SCL (SGP30 + OLED SSD1306)
GPIO 26	→	MQ-136 Heater (NPN 2N2222, R_base 1kΩ)
GPIO 27	→	MQ-131 Heater (NPN 2N2222, R_base 1kΩ)
GPIO 33	←	MQ-136 Vout (ADC1 CH5, diviseur 3,3V)
GPIO 34	←	MQ-131 Vout (ADC1 CH6 input-only)
VIN (5V)	←	Alimentation principale
GND	←	Masse commune

Règles de câblage importantes :

- Les entrées ADC GPIO 34–39 sont *input-only* : pas de pull-up interne, tension max 3,3 V.
- Lors de l'utilisation du WiFi, utiliser exclusivement **ADC1** (GPIO 32–39) — l'ADC2 est conflictuel avec le stack WiFi.
- Les signaux UART des capteurs 5 V (MH-Z19B, SDS011) passent par un **convertisseur de niveau bidirectionnel** pour protéger les GPIO 3,3 V de l'ESP32.
- Les résistances de tirage I²C (**2 × 4,7 kΩ** entre SDA/SCL et 3,3 V) sont obligatoires pour la stabilité du bus SGP30 + OLED.
- La sortie analogique des MQ-136 et MQ-131 (0–5 V) doit passer par un **diviseur résistif** (R1 = 10 kΩ, R2 = 20 kΩ) avant l'ADC pour rester ≤ 3,3 V.
- Le chauffage des capteurs MQ est alimenté en 5 V via un transistor **NPN 2N2222** (résistance de base 1 kΩ, diode 1N4007) contrôlé par les GPIO 26/27.
- Un condensateur **100 µF électrolytique** et un **100 nF céramique** sont placés en entrée/sortie du LM2596 et au plus près de chaque IC pour le découplage.
- La **masse GND est commune** à tous les modules (5 V et 3,3 V) pour garantir une référence ADC stable.

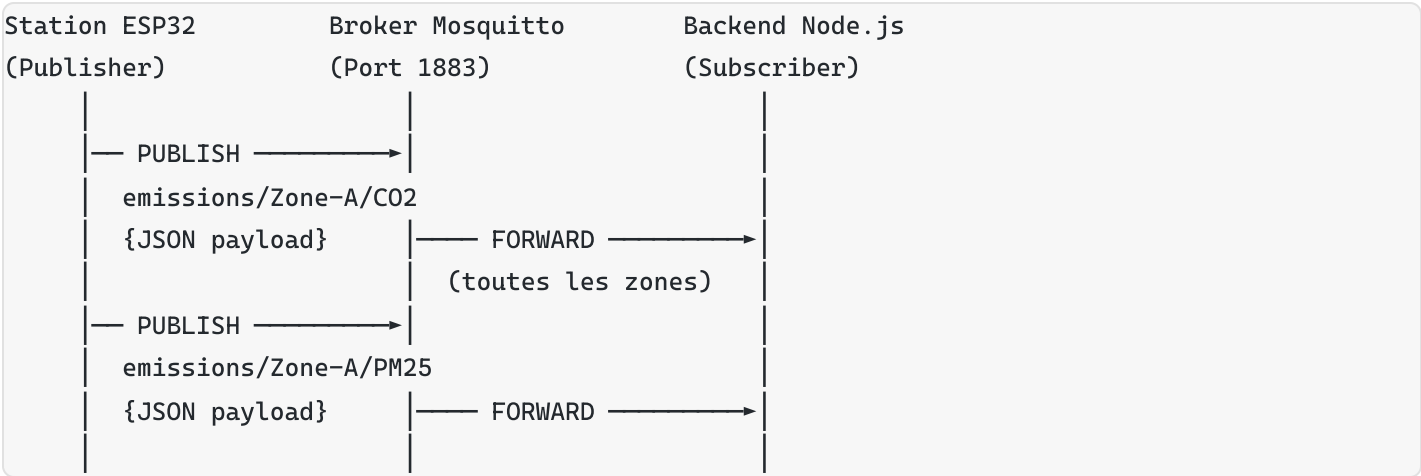
Schéma logique de la station Station-Sfax-01 en Zone-A



II.3.5 Protocole de communication MQTT

Modèle Publish/Subscribe

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie léger basé sur le modèle **Publish/Subscribe**. Ce modèle découple les producteurs de données (stations ESP32) des consommateurs (backend Node.js, tableaux de bord), permettant l'ajout de nouveaux abonnés sans modifier les stations.



Broker retenu : Mosquitto (Eclipse), version open-source, déployé en local sur le serveur backend. Port **1883** (non-TLS en environnement de développement). La configuration TLS est prévue pour la phase de production.

Structure des topics

Le schéma de topic est défini dans `config.h` :

```
emissions / <NODE_ZONE> / <sensorType>
```

Exemples de topics publiés par la station Station-Sfax-01 :

Topic	Polluant	Capteur
<code>emissions/Zone-A/CO2</code>	CO ₂	MH-Z19B
<code>emissions/Zone-A/NOX</code>	NO _x	MQ-131
<code>emissions/Zone-A/SO2</code>	SO ₂	MQ-136
<code>emissions/Zone-A/PM25</code>	PM2.5	SDS011
<code>emissions/Zone-A/PM10</code>	PM10	SDS011
<code>emissions/Zone-A/COV</code>	TVOC	SGP30
<code>emissions/Zone-A/TEMPERATURE</code>	Température	DHT22
<code>emissions/Zone-A/HUMIDITY</code>	Humidité	DHT22

Abonnement backend — wildcard : Le service `mqttService.js` s'abonne au topic `emissions/#` avec un QoS 1. Le caractère `#` est le wildcard multi-niveau MQTT : il capte tous les messages de toutes les zones et tous les types de polluants, permettant l'extension future à plusieurs stations sans modifier le backend.

```
// mqttService.js - abonnement backend
client.subscribe("emissions/#", { qos: 1 }, (err) => {
  console.log("🔔 [MQTT Service] Abonné au topic: emissions/#");
});
```

Niveaux QoS

QoS	Garantie	Choix
QoS 0	At-most-once (fire and forget)	Non retenu — perte possible
QoS 1	At-least-once (accusé de réception)	Retenu
QoS 2	Exactly-once	Surcoût inutile pour les mesures

Justification du QoS 1. Pour les mesures de pollution industrielle, la perte de données est inacceptable (risque de non-détection de dépassement). Le QoS 1 garantit la livraison avec un seul acquittement, sans le surcoût du

handshake à quatre messages du QoS 2. Les doublons éventuels (retransmission en cas de perte de l'ACK) sont gérés côté backend par la déduplication temporelle.

Format JSON du message transmis

Le payload JSON est construit dans `main.cpp` par la fonction `publishReading()` :

```
// Extrait main.cpp - construction du payload JSON
JsonDocument doc;
doc["sensorType"] = sensorType;    // Code polluant : CO2, NOX, SO2, PM25...
doc["model"]      = model;          // Référence hardware : MH-Z19B, MQ-136, MQ-131...
doc["zone"]       = NODE_ZONE;      // Zone logique : Zone-A
doc["nodeName"]   = NODE_NAME;      // Identifiant station : Station-Sfax-01
doc["value"]      = value;           // Valeur traitée (ppm, µg/m³, mg/Nm³...)
doc["rawValue"]   = rawValue;        // Valeur brute (tension TIA en V pour ME4, ppb pour SGP30)
doc["unit"]       = unit;            // Unité : ppm, µg/m³, mg/Nm³, °C, %RH
doc["level"]      = level;           // Niveau indicatif local : normal|warning|high|critical
doc["timestamp"]  = isoTimestamp();  // ISO 8601 UTC (NTP)
doc["isValid"]    = isValid;         // false si capteur en défaut
doc["rssi"]       = WiFi.RSSI();     // Signal WiFi en dBm (diagnostic)
doc["battery"]    = nullptr;         // null : alimentation secteur permanente
```

Exemple de message CO₂ publié sur `emissions/Zone-A/CO2` :

```
{
  "sensorType": "CO2",
  "model": "MH-Z19B",
  "zone": "Zone-A",
  "nodeName": "Station-Sfax-01",
  "value": 850,
  "rawValue": 850,
  "unit": "ppm",
  "level": "warning",
  "timestamp": "2026-06-04T08:32:15Z",
  "isValid": true,
  "rssi": -62,
  "battery": null
}
```

Exemple de message SO₂ publié sur `emissions/Zone-A/SO2` :

```
{
  "sensorType": "S02",
  "model": "MQ-136",
  "zone": "Zone-A",
  "nodeName": "Station-Sfax-01",
  "value": 12.01,
  "rawValue": 0.336,
  "unit": "mg/Nm³",
  "level": "normal",
  "timestamp": "2026-06-04T08:32:45Z",
  "isValid": true,
  "rssi": -62,
  "battery": null
}
```

`value` = $4,2 \text{ ppm} \times 2,86 = \mathbf{12,01 \text{ mg/Nm}^3}$ (post-conversion firmware). `rawValue` = tension TIA en volts (0,336 V), utile pour le diagnostic du circuit de conditionnement.

Exemple de message PM2.5 publié sur `emissions/Zone-A/PM25` :

```
{
  "sensorType": "PM25",
  "model": "SDS011",
  "zone": "Zone-A",
  "nodeName": "Station-Sfax-01",
  "value": 38.7,
  "rawValue": 38.7,
  "unit": "µg/m³",
  "level": "high",
  "timestamp": "2026-06-04T08:32:30Z",
  "isValid": true,
  "rssi": -62,
  "battery": null
}
```

II.3.6 Stratégie de transmission et fréquences de publication

Boucle principale non bloquante

Le firmware adopte une architecture de planification **non bloquante** basée sur `millis()`. La boucle `loop()` vérifie en permanence si l'intervalle de chaque famille de capteurs est écoulé, sans aucun `delay()` bloquant dans le chemin critique.

```
// Extrait main.cpp - loop() non bloquant
void loop() {
    // Maintien connexions réseau
    if (WiFi.status() != WL_CONNECTED) connectWiFi();
    if (!mqtt.connected()) connectMqtt();
    mqtt.loop(); // Keep-alive MQTT et callbacks

    const unsigned long now = millis();

    if (now - lastCo2Ms ≥ INTERVAL_CO2_MS) { // 10 000 ms
        lastCo2Ms = now;
        publishCo2();
    }
    if (now - lastPmMs ≥ INTERVAL_PM_MS) { // 15 000 ms
        lastPmMs = now;
        publishParticulates();
    }
    if (now - lastGasMs ≥ INTERVAL_GAS_MS) { // 30 000 ms
        lastGasMs = now;
        publishGases();
    }
    if (now - lastEnvMs ≥ INTERVAL_ENV_MS) { // 10 000 ms
        lastEnvMs = now;
        publishEnvironment();
    }
    delay(50); // Cède au watchdog et au stack WiFi
}
```

Fréquences de publication par capteur

Les fréquences sont définies dans `config.h` et justifiées par les caractéristiques physiques de chaque capteur :

Capteur	Intervalle	Justification
MH-Z19B (CO ₂)	10 s	Réponse UART rapide, capteur NDIR sans contrainte de chauffe en régime permanent
DHT22 (T°/HR)	10 s	Temps de conversion ~2 s, limite datasheet : 0,5 Hz maximum
SDS011 (PM2.5/PM10)	15 s	Cycle d'acquisition laser + paquet UART + stabilisation ventilateur
MQ-136 + MQ-131 + SGP30 (SO ₂ /NO _x /COV)	30 s	Moyennage ADC sur 16 échantillons (MQ), stabilisation I ² C SGP30 (T90 ≤ 30 s)

Charge réseau estimée. À ces fréquences, la station génère environ :

- 3 messages par 30 s pour SO₂, NO₂ et COV
- 2 messages par 15 s pour PM
- 4 messages par 10 s pour CO₂ + T/HR
- Soit **≈ 12 messages/minute**, chacun de ~384 octets → **≈ 4,6 KB/min** de trafic MQTT montant

Cette charge est largement compatible avec une connexion WiFi industrielle standard.

Cohérence avec le simulateur backend

Les intervalles du firmware sont alignés sur ceux du simulateur Node.js (`iot/simulator.js`) pour assurer la compatibilité des données lors des phases de test sans hardware. Le backend `ReadingService` traite indifféremment les messages provenant du firmware ESP32 ou du simulateur, car le format JSON est identique.

II.3.7 Gestion de la fiabilité et reconnexion

Reconnexion automatique MQTT et WiFi

Le firmware implémente une stratégie de reconnexion robuste pour les deux couches réseau. La reconnexion est tentée à chaque itération de `loop()` si la connexion est perdue, avec un délai de **2 000 ms entre les tentatives MQTT** (configurable via `reconnectPeriod` côté backend) :

```
// Extrait main.cpp – reconnexion MQTT
static void connectMqtt() {
    if (mqtt.connected()) return;
    mqtt.setServer(MQTT_BROKER, MQTT_PORT);
    mqtt.setBufferSize(512);
    while (!mqtt.connected()) {
        Serial.printf("[MQTT] Connexion %s:%d ... \n", MQTT_BROKER, MQTT_PORT);
        if (mqtt.connect(MQTT_CLIENT_ID)) {
            Serial.println("[MQTT] Connecté");
            return;
        }
        Serial.printf("[MQTT] Échec rc=%d\n", mqtt.state());
        delay(3000); // Attente avant nouvelle tentative
    }
}
```

Côté backend, le service `mqttService.js` est configuré avec `reconnectPeriod: 2000` (2 s) pour la reconnexion automatique au broker en cas de coupure.

```
// mqttService.js - reconnexion backend
const client = mqtt.connect(MQTT_BROKER, {
  clientId: "pollution-backend-" + Math.random().toString(16).slice(2, 8),
  keepalive: 60,
  reconnectPeriod: 2000, // Reconnexion toutes les 2 secondes
});

client.on("reconnect", () => {
  console.log("🔌 [MQTT Service] Reconnexion ... ");
});
```

Validation côté firmware — flag `isValid`

Avant toute publication, chaque lecture de capteur est accompagnée d'un indicateur de validité `valid` (struct `GasReading`, `PmReading`, `EnvReading` dans `sensors.h`). Une lecture est marquée invalide si :

Condition	Capteur concerné
Tension ADC < 0,05 V (capteur débranché ou court-circuit)	MQ-136, MQ-131
Tension ADC > 3,20 V (diviseur résistif absent ou défaillant)	MQ-136, MQ-131
Retour CO ₂ = 0 (erreur UART ou capteur non prêt)	MH-Z19B
Trame SDS011 incorrecte (<code>result.isOk() = false</code>)	SDS011
<code>isnan(temperature)</code> ou <code>isnan(humidity)</code>	DHT22
Initialisation SGP30 échouée (<code>sgp30.begin() = false</code>)	SGP30

Le champ `isValid` est propagé dans le payload JSON. Côté backend, `mqttService.js` passe ce champ à `ReadingService` qui le stocke dans MongoDB :

```
// mqttService.js - propagation isValid
const readingPayload = {
  ...
  isValid: data.isValid !== false, // false si capteur en défaut
  ...
};
```

Validation côté backend — rejet des valeurs physiquement impossibles

`ReadingService.ingestReading()` applique une deuxième couche de validation métier : toute valeur supérieure à **10 fois la limite réglementaire** du polluant est rejetée comme erreur capteur :

```
// ReadingService.js - validation métier
const validityMax = polluant.regulatoryLimit
  ? polluant.regulatoryLimit * 10
  : 10000;
const isValid = value ≥ 0 && value ≤ validityMax;
```

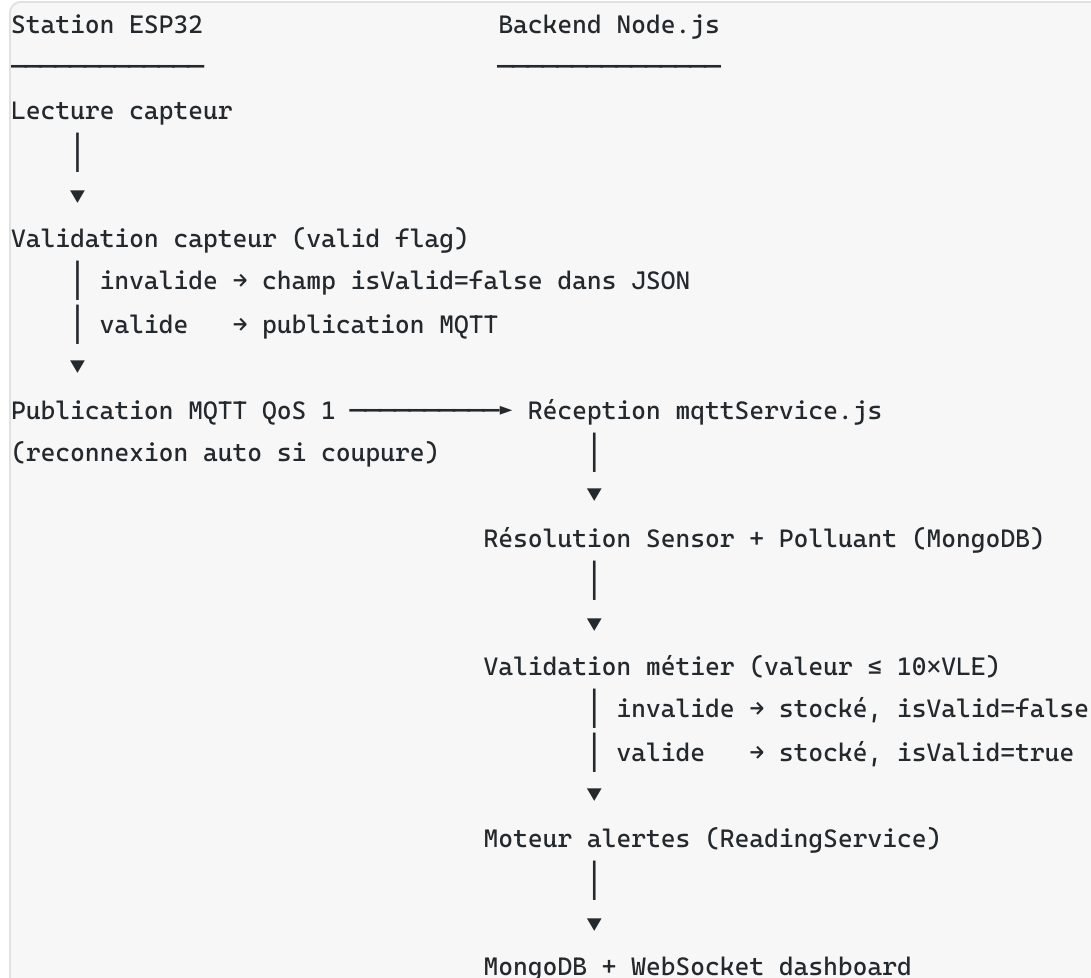
Exemple appliqué : Pour le SO₂ dont la VLE générale est 300 mg/Nm³, toute valeur > 3 000 mg/Nm³ est rejetée automatiquement (`isValid = false`). Pour le NO₂ (VLE 500 mg/Nm³), tout dépassement de 5 000 mg/Nm³ déclenche le rejet. Cela évite que des tensions TIA anormales (saturation du circuit op-amp, court-circuit) ne génèrent de fausses alertes critiques.

Horodatage NTP

Les timestamps ISO 8601 UTC sont générés par `isoTimestamp()` après synchronisation NTP au démarrage (`configTime(0, 0, "pool.ntp.org", "time.nist.gov")`). En cas d'échec NTP, la fonction retourne une chaîne vide et le backend utilise `new Date()` comme fallback :

```
// mqttService.js - fallback timestamp
timestamp: data.timestamp || new Date(),
```

Architecture de fiabilité — vue d'ensemble



Cette double validation (firmware + backend) constitue une défense en profondeur qui garantit la qualité des données stockées et la pertinence des alertes générées, conformément aux exigences d'un système de surveillance industrielle.